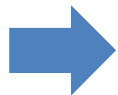


学习内容

CONTENTS



1

机器学习概述

2

线性回归与逻辑回归

3

支持向量机

4

聚类与降维

5

决策树

6

强化学习

模型/方法	学习类型	任务类型
线性回归	监督学习	回归
逻辑回归	监督学习	分类
决策树	监督学习	分类、回归
随机森林	监督学习	分类、回归 (集成学习)
SVM	监督学习	分类、回归 (SVR)
KNN	监督学习	分类、回归
贝叶斯算法 (如朴素贝叶斯)	监督学习	分类
神经网络	监督/无监督/强化学习	分类、回归、降维、生成数据、强化学习
集成学习 (如随机森林、梯度提升)	监督学习	分类、回归
强化学习算法 (Q-learning、策略梯度)	强化学习	动态决策任务
K-均值算法	无监督学习	聚类



1、什么是回归分析



2、什么是线性回归



3、线性回归实现的基本步骤



4、线性回归实例

5、什么是逻辑回归

6、逻辑回归实例



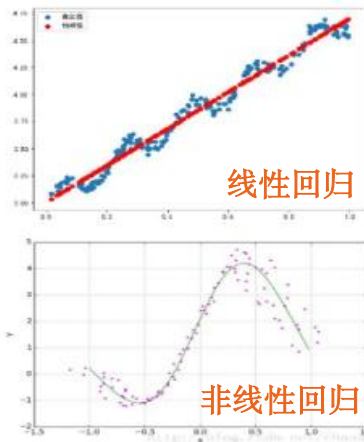
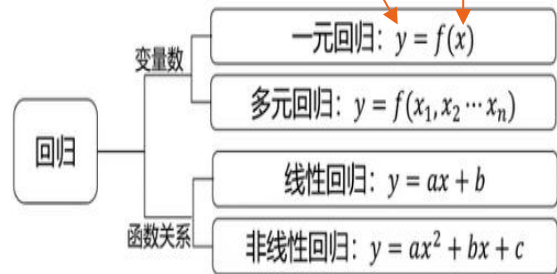
XX SCHOOL



回归分析:分析两种或两种以上变量间的定量关系

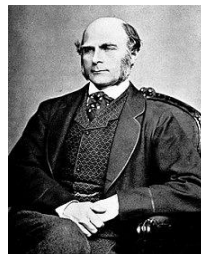
现实生活中,需要分析如碳排放量与气候变暖之间的关系,某一商品广告投入量与该商品销售量之间的关系等,这种分析不同变量之间存在关系的研究叫作**回归分析**,刻画不同变量之间关系的模型称为**回归模型**。

因变量 自变量



英国著名生物学家兼统计学家
高尔顿

Sir Francis Galton (1822-1911)



$$y = 33.73(\text{英寸}) + 0.516x$$

y: 子女平均身高

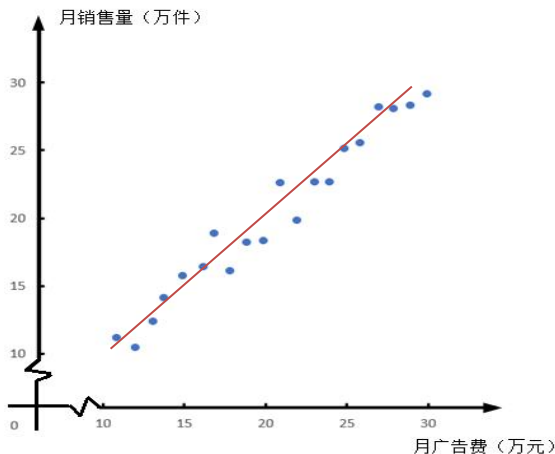
x: 父母平均身高

• 回归分析还可根据因变量数量分为: 简单回归分析/多重回归分析

- 父母平均身高每增加一个单位, 其成年子女平均身高只增加0.516个单位, 它反映了这种“衰退(regression)”效应 (“回归”到正常人平均身高)。
- 虽然x和y之间并不总是具有“衰退”(回归)关系, 但是“回归”这一名称保留了下来。

【例题1】 某企业某商品月广告费用与月销售量数据如表所示:

i (月份)	1	2	3	4	5	6	7	8	9	10
s_i (月广告费 万元)	10.95	12.14	13.22	13.87	15.06	16.30	17.01	17.93	19.01	20.01
t_i (月销售量 万件)	11.18	10.43	12.36	14.15	15.73	16.40	18.86	16.13	18.21	18.37
i (月份)	11	12	13	14	15	16	17	18	19	20
s_i (月广告费 万元)	21.04	22.10	23.17	24.07	25.00	25.95	27.10	28.01	29.06	30.05
t_i (月销售量 万件)	22.61	19.83	22.67	22.70	25.16	25.55	28.21	28.12	28.32	29.18

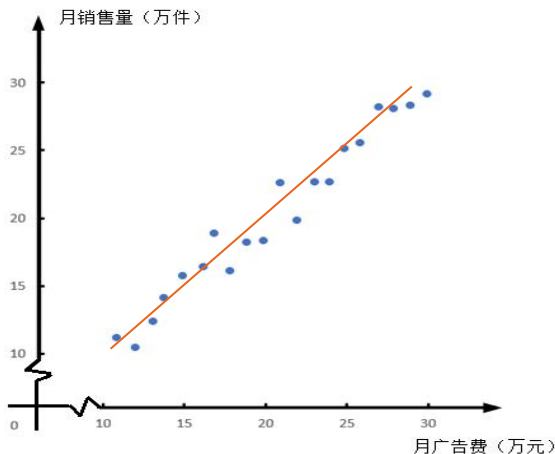


月广告费用 s_i 与月销售量数据 t_i 之间的关系可表示:

$$t = f(s) = as + b$$

【例题1】 某企业某商品月广告费用与月销售量数据如表所示：

i (月份)	1	2	3	4	5	6	7	8	9	10
s_i (月广告费 万元)	10.95	12.14	13.22	13.87	15.06	16.30	17.01	17.93	19.01	20.01
t_i (月销售量 万件)	11.18	10.43	12.36	14.15	15.73	16.40	18.86	16.13	18.21	18.37
i (月份)	11	12	13	14	15	16	17	18	19	20
s_i (月广告费 万元)	21.04	22.10	23.17	24.07	25.00	25.95	27.10	28.01	29.06	30.05
t_i (月销售量 万件)	22.61	19.83	22.67	22.70	25.16	25.55	28.21	28.12	28.32	29.18



s_i 与 t_i 之间的关系模型可表示：

$$t = f(s) = as + b$$

学习准则：MSE (误差平方和的均值)

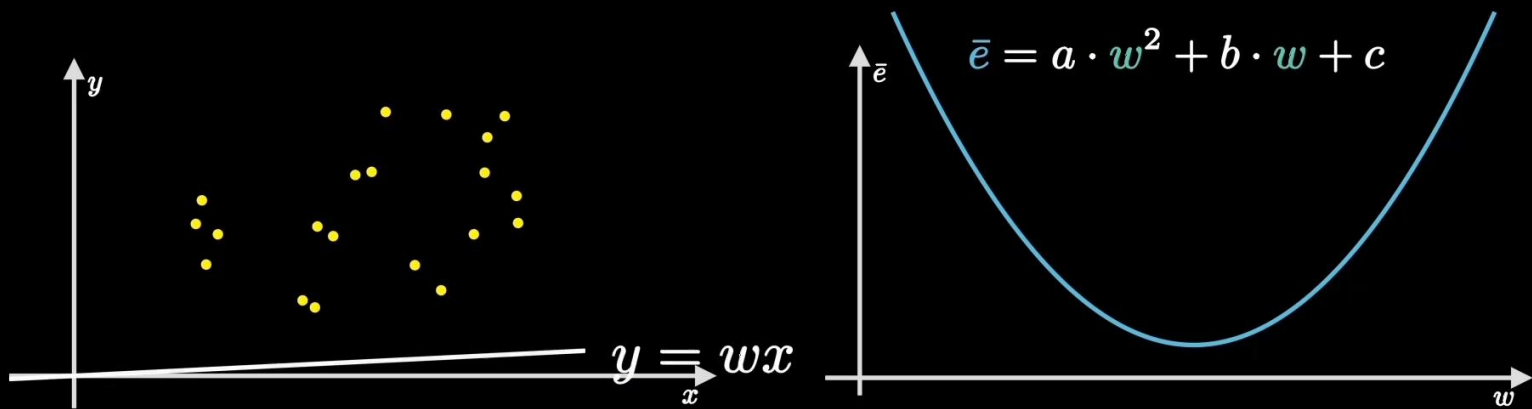
优化算法：最小二乘法/梯度下降法

求得模型参数： $a = 0.968$, $b = 0.191$

得到线性回归模型：

$$f(s) = 0.968s + 0.191$$

梗直哥讲AI bilibili

减小误差 $\bar{e}$

- **损失函数 (Loss Function)**：度量单个样本预测的错误程度，损失函数值越小，模型就越好

- **平方损失**：对于单个样本，模型的预测值和真实值（标签）之间误差的平方，常用于回归问题

$$Loss(y_i, f(x_i)) = (y_i - f(x_i))^2$$

- **交叉熵损失**：衡量模型预测的概率分布与真实分布之间的偏差程度，常用于分类问题

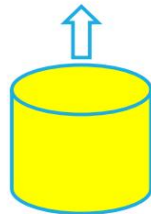
$$Loss(y_i, P(y_i|x_i)) = -\log P((y_i|x_i))$$

- **代价函数 (Cost Function)**：在整个训练集上，所有样本误差的平均，也就是损失函数的平均，即经验风险最小化函数。

- **目标函数 (Object Function)**：最终需要优化的函数，等于经验风险 + 结构风险（也就是 Cost Function + 正则化项）。

训练映射函数 f

使得 $f(x_i)$ 预测结果尽量等于 y_i



训练数据集

$(x_i, y_i), i = 1, \dots, n$

- 线性回归(linear regression): 拟合函数是一次函数，例如：

$$f(x_0, x_1, x_2, \dots) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3 \dots + \dots$$

其中 $x_1, x_2, \dots$ 是训练样本集中样本的各个特征(feature), $w_0, w_1, w_2, \dots$ 是模型的未知参数。 w_i 表示第 i 个特征 x_i 对模型输出的影响程度。

x_1	x_2	x_3	x_4	
面积	楼层	朝向	有地铁	



线性回归：通过属性特征的线性组合来进行预测的**线性模型**，其目的是找到一条直线或者一个平面或者更高维的超平面，使得**预测值与真实值之间的误差最小**。

- x_1 表示面积，房价预测模型可用一个线性拟合函数：

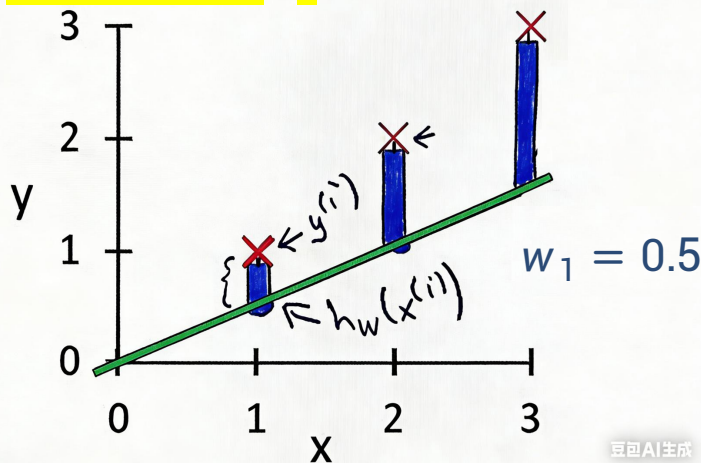
$$h_w(x) = w_0 + w_1 x_1 = [w_0 \quad w_1] \begin{bmatrix} 1 \\ x_1 \end{bmatrix} = w^T x$$



- 对 $h_w(x)$ 函数进行评估，一般评估函数称为损失函数，采用平方损失，要找到一组参数 w (这里是 w_0, w_1)，使得均方误差 $J(w)$ 最小：

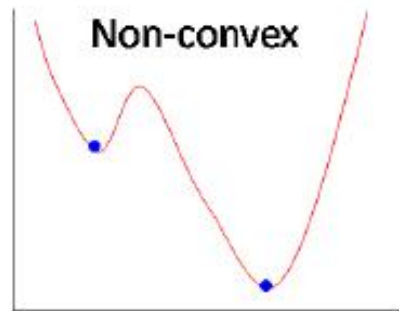
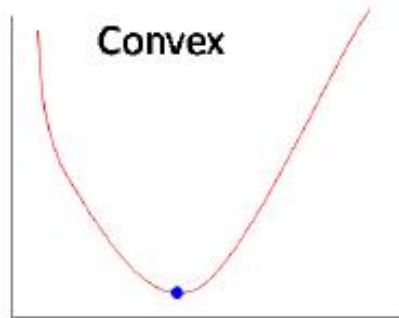
$$J(w) = \frac{1}{2m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)})^2$$

m : 样本数
 第 i 个样本的特征
 第 i 个样本的预测值
 第 i 个样本的真实值



- 机器学习问题转化成为一个最优化问题: 找到一组参数 w ，使得目标函数 $J(w)$ 最小

$$\min_w J(w)$$



- 如何调整 w 以使得目标函数 $J(w)$ 取得最小值有很多方法，其中有最小二乘法(将最优问题转化为求函数极值问题)，是一种完全是数学描述的经典方法。

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & x_3^{(1)} & \dots & x_n^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & x_3^{(2)} & \dots & x_n^{(2)} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_1^{(m)} & x_2^{(m)} & x_3^{(m)} & \dots & x_n^{(m)} \end{bmatrix} \quad Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \dots \\ y^{(m)} \end{bmatrix}$$

当矩阵 X 的行向量之间存在一定的线性相关性时，就会使得矩阵 $X^T X$ 不可逆。

$$\text{矩阵} X \text{满秩: } \mathbf{w} = (X^T X)^{-1} X^T Y$$

m 代表训练集中样本的数量， n 代表特征的数量， $\mathbf{x}$ 代表特征/输入变量， y 代表目标变量/输出变量，

$(\mathbf{x}, y)$ 代表训练集中的样本， $(\mathbf{x}^{(i)}, y^{(i)})$ 代表第 i 个观察样本

● **梯度下降法**是解决回归模型中最简单且有效的方法之一,当特征数量 n 大时也能较好适用,适用于各种类型的模型。

1) 首先对初始的 w 赋值,可以随机,也可以为一个全零的向量。

2) 改变 w 的值,使得 $J(w)$ 沿着**负梯度方向**进行减少。

$$h_w(x) = w^T x$$

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)})^2$$

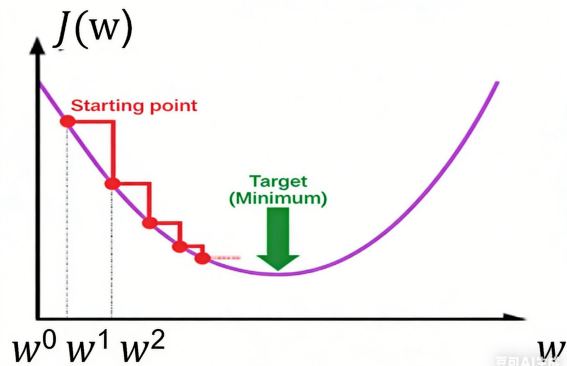
第 i 个样本
的真实值

第 i 个样本
的预测值

$$w^{t+1} = w^t - \alpha \frac{\partial}{\partial w} J(w)$$

学习率
/步长

参数梯度



- 迭代方向: **目标函数 $J(w)$ 值下降最快的方向——负梯度方向**
- 迭代步长: **合适的长度**

- 求参数梯度，即对损失函数 $J(w)$ 求偏导：

$$\frac{\partial}{\partial w} J(w) = \frac{\partial}{\partial w} \frac{1}{2m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)})^2 = \frac{1}{m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)}) x^{(i)}$$

 真实
值

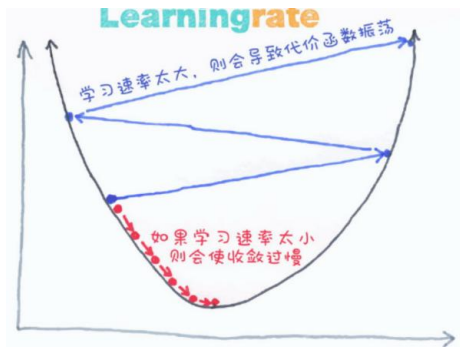
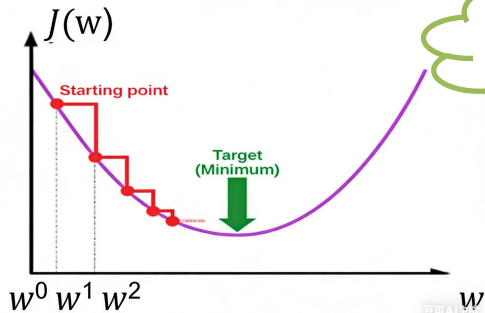
$$h_w(x) = w^T x$$

- 更新的过程

 学习率
/步长

 预测
值

$$w^{t+1} = w^t - a \frac{\partial}{\partial w} J(w) = w^t - a \frac{1}{m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)}) x^{(i)}$$

 学习率越大越好
还是越小越好？

 参数
梯度

- 求参数梯度，即对损失函数 $J(w)$ 求偏导：

$$h_w(x) = w^T x$$

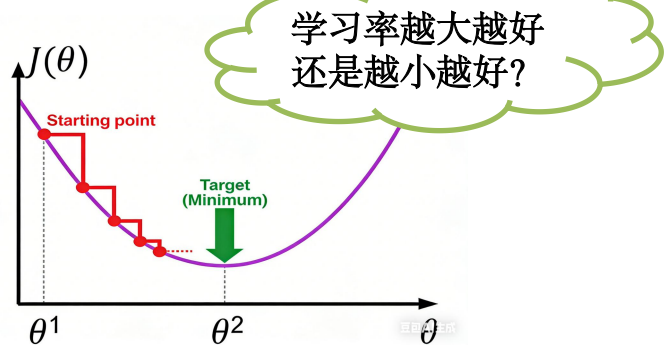
$$\frac{\partial}{\partial w} J(w) = \frac{\partial}{\partial w} \frac{1}{2m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)})^2 = \frac{1}{m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)}) x^{(i)}$$

- 更新的过程

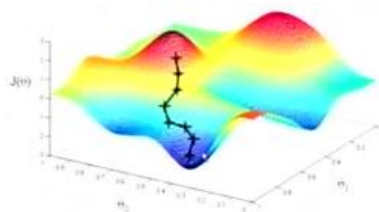
BGD: 梯度下降的每一步中，用到了所有的训练样本

$$w^{t+1} = w^t - a \frac{\partial}{\partial w} J(w) = w^t - a \frac{1}{m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)}) x^{(i)}$$

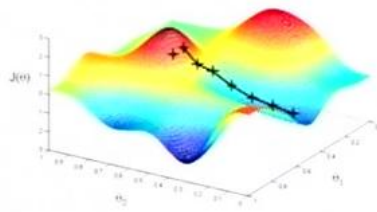
参数
梯度



Gradient Descent



Gradient Descent



关于学习率的选择，以下哪种说法最准确？

- ☐ A 学习率越大，收敛速度越快，因此应尽可能选择大的学习率
- ☐ B 学习率越小越好，虽然慢但一定能收敛
- ☐ C 学习率过大可能导致无法收敛，过小则收敛缓慢，需要合理设置
- ☐ D 学习率不影响梯度下降的结果，可以任意设定

提交

关于学习率的选择，以下哪种说法最准确？

- ☐ A 学习率越大，收敛速度越快，因此应尽可能选择大的学习率
- ☐ B 学习率越小越好，虽然慢但一定能收敛
- ☒ C 学习率过大可能导致无法收敛，过小则收敛缓慢，需要合理设置
- ☐ D 学习率不影响梯度下降的结果，可以任意设定

提交

每次迭代使用一个样本来对参数进行更新

$$h_w(x) = w^T x$$

- 求参数梯度，对损失函数 $J(\theta)$ 求偏导：

$$\frac{\partial}{\partial w} J(w) = \frac{\partial}{\partial w} \frac{1}{2} (h_w(x^{(i)}) - y^{(i)})^2 = (h_w(x^{(i)}) - y^{(i)}) x^{(i)}$$

- 更新的过程

学习
率

预测
值

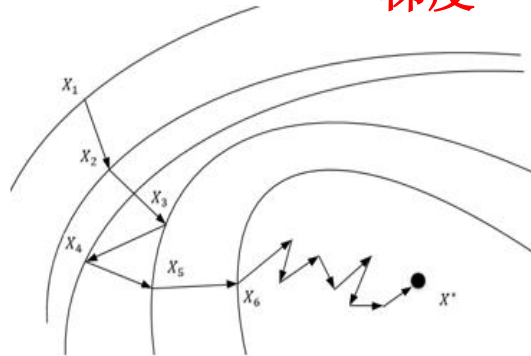
真实
值

$$w^{t+1} = w^t - a \frac{\partial}{\partial w} J(w) = w^t - a (h_w(x^{(i)}) - y^{(i)}) x^{(i)}$$

参数
梯度

SGD: 梯度下降的每一步中，用到一个样本，梯度计算快，降低时间成本；更新过程存在一定随机性，并非每一次更新都会使得目标函数值下降，但整体呈下降趋势，并具备跳出局部最优的能力。

◆ 小批量随机梯度下降法：每次抽取训练集中部分样本参与训练。



关于随机梯度下降法（SGD），下列说法正确的是？

- ☐ A 每次迭代使用整个训练集来计算梯度
- ☐ B SGD的每一次参数更新都会使损失函数值严格下降
- ☐ C SGD通过使用单个样本来估计梯度，降低了计算成本
- ☐ D SGD无法跳出局部最优解

提交

关于随机梯度下降法（SGD），下列说法正确的是？

- ☐ A 每次迭代使用整个训练集来计算梯度
- ☐ B SGD的每一次参数更新都会使损失函数值严格下降
- ☒ C SGD通过使用单个样本来估计梯度，降低了计算成本
- ☐ D SGD无法跳出局部最优解

提交

线性回归扩展算法：用简单的**基函数 $\Phi(\mathbf{x})$** 替换输入变量 $\mathbf{x}$ 。这样线性拟合形式扩展到了**固定非线性函数的线性组合**。

$$h_w(\mathbf{x}) = w_0 + w_1x_1 + \dots + w_dx_d = w^T\mathbf{x}$$



$$h_w(\mathbf{x}) = w_0 + \sum_{j=1}^d w_j x_j^j$$

多项式拟合取 $\phi_j(x_i) = x_i^j$

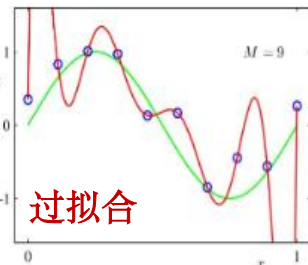
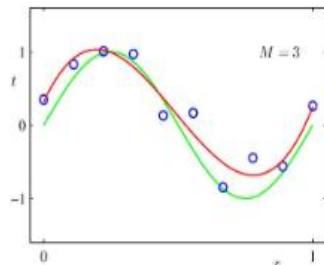
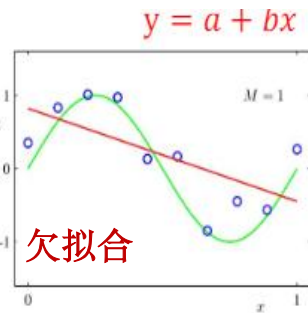
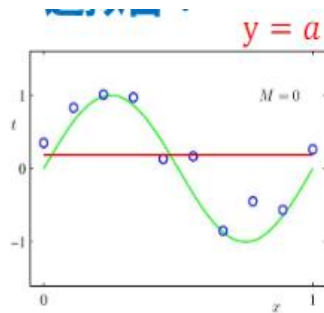
$$h_w(\mathbf{x}) = w_0 + w_1x_i + w_2x_i^2 + \dots + w_dx_i^d = w_0 + \sum_{j=1}^d w_j x_i^j = w^T \phi(\mathbf{x})$$

$$J(w) = \frac{1}{2} \sum_{k=1}^m (h_w(x^{(k)}) - y^{(i)})^2$$

预测
值

真实
值

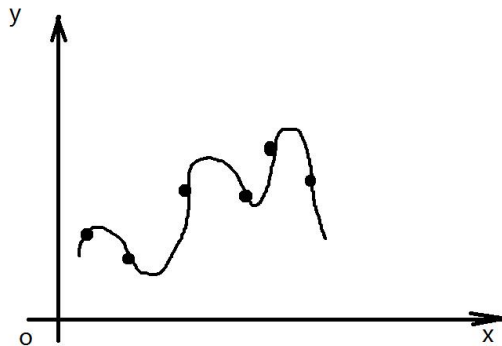
一元线性回归
(线性拟合)



$$y = ax^3 + bx^2 + cx + d$$

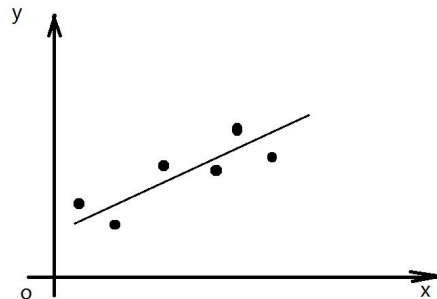
一元多项式回归
(非线性拟合)

优化 正则化

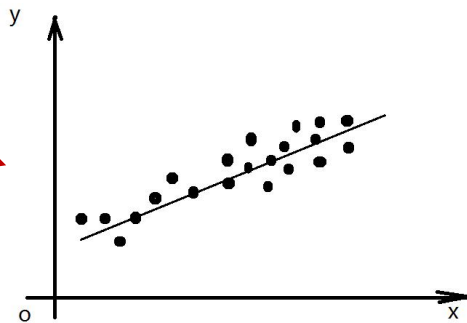


3.正则化: 目的在于提升模型泛化能力

2.减小模型复杂度



1.增加训练样本



存在噪音或样本少, 模型复杂, 过拟合

$$L_1 \text{ 正则化: } J(w) = \frac{1}{2m} \sum_{i=1}^m (\underbrace{h(x^{(i)})}_{\text{预测值}} - \underbrace{\hat{y}^{(i)}}_{\text{真实值}})^2$$

$$+ \lambda \sum_{j=1}^n |w_j|, \text{ Lasso Regression (Lasso回归)}$$

$$L_2 \text{ 正则化: } J(w) = \frac{1}{2m} \sum_{i=1}^m (h(x^{(i)}) - \hat{y}^{(i)})^2$$

$$+ \lambda \sum_{j=1}^n w_j^2, \text{ Ridge Regression (岭回归)}$$

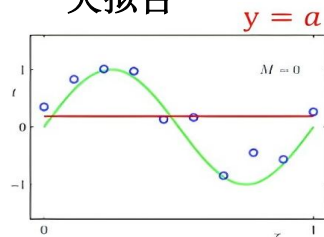
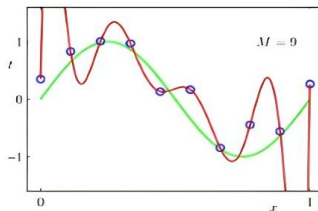
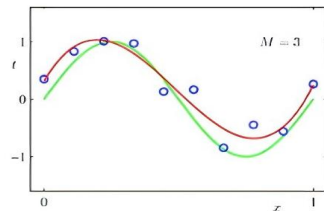
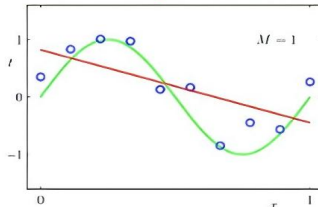
$$\text{(弹性网络): } J(w) = \frac{1}{2m} \sum_{i=1}^m (h(x^{(i)}) - \hat{y}^{(i)})^2 +$$

$$\lambda(\rho \cdot \sum_{j=1}^n |w_j| + (1 - \rho) \cdot \sum_{j=1}^n w_j^2)$$

- λ 为正则化系数, 调整正则化项与训练误差的比例, $\lambda > 0$ 。
- $1 \geq \rho \geq 0$ 为比例系数, 调整 L_1 正则化与 L_2 正则化的比例。

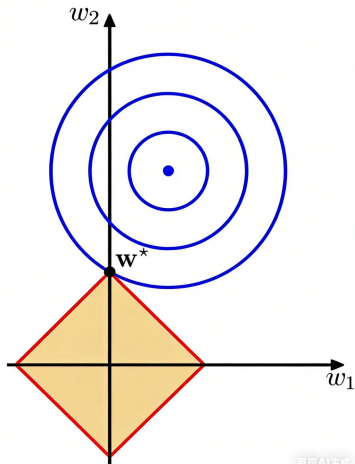
 线性回归
(线性拟合)

欠拟合

 $y = a + bx$  $y = ax^3 + bx^2 + cx + d$

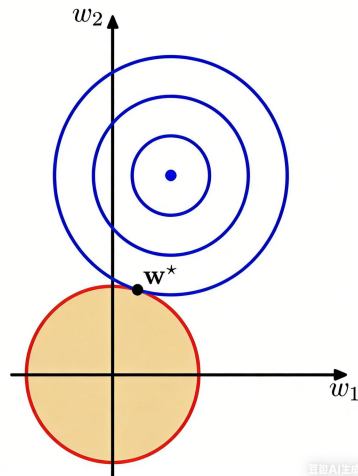
过拟合

 多项式回归
(非线性拟合)



L1正则化是指在损失函数中加入权值向量 $\mathbf{w}$ 的绝对值之和，L1的功能是使权重稀疏。

L1正则化可以产生稀疏模型



L2正则化是在损失函数中加入权值向量 $\mathbf{w}$ 的平方和，L2的功能是使权重平滑。

L2正则化可以防止过拟合

图上面中的蓝色轮廓线是没有正则化损失函数的等高线，中心的蓝色点为最优解。

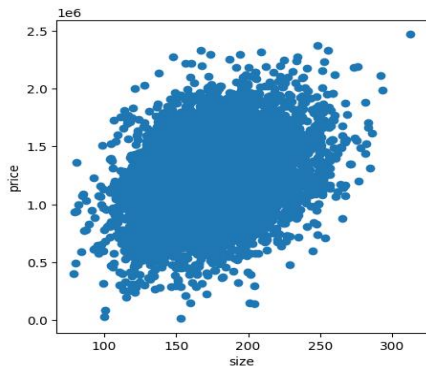
L1正则化给出的最优解 $\mathbf{w}^*$ 是使解更加靠近原点，也就是说L2正则化能降低参数范数的总和。

L1正则化给出的最优解 $\mathbf{w}^*$ 是使解更加靠近某些轴，而其它的轴则为0，所以L1正则化能使得到的参数稀疏化。

实战2-1：单因子房价预测

- 基于 **usa_housing_price_single.csv** 数据（5000组样本数据，存放于csv文件），以房屋面积（size）作为输入变量，来预测房屋价格，属于**一元线性回归**
- 线性方程： $y=ax+b$ （房屋价格= a ×房屋面积+ b ）

	size	Price
0	188.214212	1.059034e+06
1	160.042526	1.505891e+06
2	227.273545	1.058988e+06
3	164.816630	1.260617e+06
4	161.966659	6.309435e+05



实战2-1：单因子房价预测

#1、导入所需的库

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from matplotlib import pyplot as plt
import numpy as np
import pandas as pd
```

- NumPy：Python科学计算的核心库，专注于高效多维数组运算和数值计算
- Pandas：基于NumPy构建的数据处理工具，擅长结构化数据分析与清洗

实现基本步骤

1、导入所需的库

实战2-1：单因子房价预测

#2、准备数据集

```
data = pd.read_csv('usa_housing_price_single.csv')
```

```
x=data.loc[:, 'size']
```

```
y=data.loc[:, 'Price']
```

#可视化

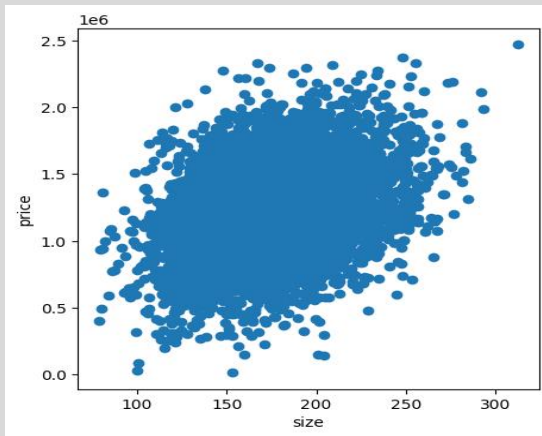
```
fig1 = plt.figure(figsize=(5,5))
```

```
plt.scatter(x,y)
```

```
plt.xlabel('size')
```

```
plt.ylabel('price')
```

```
plt.show()
```



实现基本步骤

1、导入所需的库

2、准备数据集

实战2-1：单因子房价预测

#调整数据格式

```
x = np.array(x)
```

```
x = x.reshape(-1,1)
```

```
print(x.shape)
```

```
(5000, 1)
```

3. 建构并训练模型

```
lr = LinearRegression() #使用LinearRegression构建线性回归模型
```

```
lr.fit(x,y) #训练模型
```

#x: 特征数据（必需），形状为 (n_samples, n_features) 的数组或稀疏矩阵

#y: 目标值（必需），形状为 (n_samples,) 或 (n_samples, n_targets) 的数组

实现基本步骤

1、导入所需的库

2、准备数据集

3、建构并训练模型

实战2-1：单因子房价预测

4. 查看模型参数

```
print('特征系数a为:',lr.coef_)  
print('截距b为:',lr.intercept_)
```

特征系数a为: [3674.54995744]
截距b为: 585279.3313038232

5. 评估模型性能

```
y_predict=lr.predict(x)  
print(y_predict)
```

```
[1276881.85636623 1173363.58767144 1420407.32457443 ... 1097848.86467426  
1264502.88144558 1131278.58816273]
```

```
MSE = mean_squared_error(y,y_predict)
```

```
R2 = r2_score(y,y_predict)
```

```
print(MSE, R2)
```

```
108771672553.6264 0.1275031240418234
```

实现基本步骤

- 1、导入所需的库
- 2、准备数据集
- 3、建构并训练模型
- 4、查看模型参数
- 5、评估模型性能

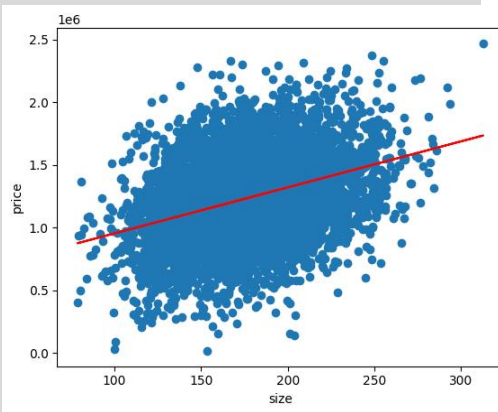
实战2-1：单因子房价预测

6. 可视化模型与数据点

```
fig2 = plt.figure()
plt.scatter(x, y) #绘制散点图
plt.plot(x, y_predict, color= 'red' ) #绘制线图
plt.xlabel('size') #添加图例
plt.ylabel('prize')
plt.show()
```

7. 预测新数据

```
x_new=[ [80], [500] ]
y_new_predict=lr.predict(x_new)
print(f'80平方和500平方的预测值为：{y_new_predict[0]}, {y_new_predict[1]}')
```



实现基本步骤

- 1、导入所需的库
- 2、准备数据集
- 3、建构并训练模型
- 4、查看模型参数
- 5、评估模型性能
- 6、可视化模型与数据点
- 7、预测新数据

实战2-2：多因子房间预测

- 基于usa_housing_price.csv数据（5000组样本数据，存放于csv文件），以人均收入(Avg.Area Income)、房屋年龄(Avg.Area House Age)、房间数量(Avg.Area Numbers of rooms)、区域人口(Area poplution)、房屋面积(size)作为输入变量，来预测房屋价格，属于多元线性回归
- 线性方程：

$$y=a_1x_1+a_2x_2+a_3x_3+a_4x_4+a_5x_5+b$$

- 研究多个自变量（输入变量）与因变量（输出变量）之间的线性关系，找到合适的模型 a_i 和 b ，使得损失函数的值最小

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Area Population	size	Price
0	79545.45857	5.317139	7.009188	23086.80050	188.214212	1.059034e+06
1	79248.64245	4.997100	6.730821	40173.07217	160.042526	1.505891e+06
2	61287.06718	5.134110	8.512727	36882.15940	227.273544	1.058988e+06
3	63345.24005	3.811764	5.586729	34310.24283	164.816630	1.260617e+06
4	59982.19723	5.959445	7.839388	26354.10947	161.966659	6.309435e+05

实战2-2：多因子房价预测

#1、导入所需的库

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from matplotlib import pyplot as plt
import numpy as np
import pandas as pd
```

#2、准备数据集

```
data = pd.read_csv('usa_housing_price.csv')
x=data.drop('Price', axis=1) # 使用drop去除data数据中的price
y=data.loc[:, 'Price']
```

实现基本步骤

1、导入所需的库

2、准备数据集

3、建构并训练模型

4、查看模型参数

5、评估模型性能

6、可视化模型与数据点

7、预测新数据

实战2-2：多因子房价预测

#可视化

```
plt.figure(figsize=(10,10))
```

```
fig1 = plt.subplot(231)
```

```
plt.scatter(x.loc[:, 'Avg. Area Income'], y)
```

```
plt.title('Price VS Income')
```

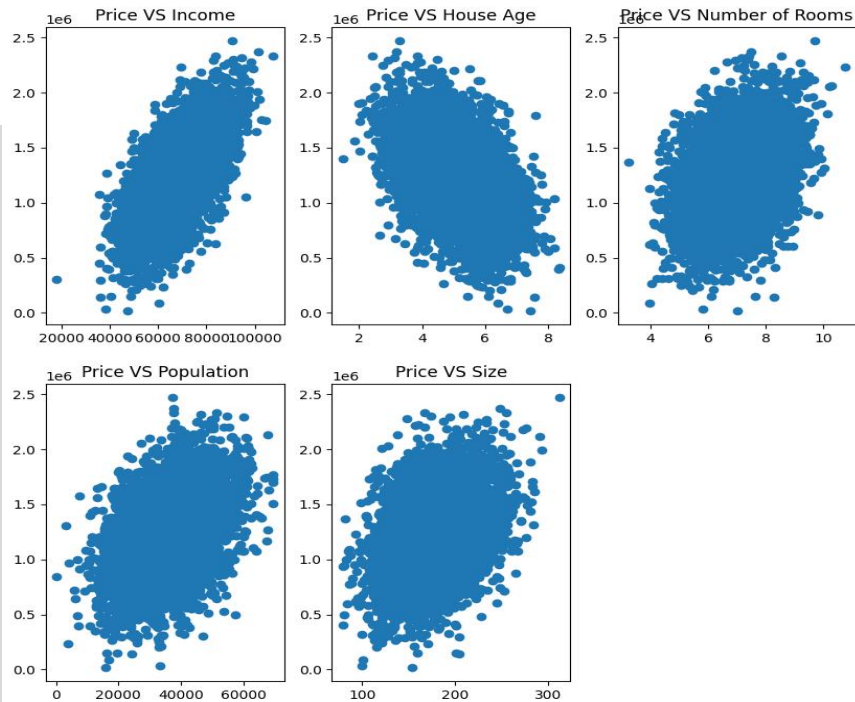
```
fig2 = plt.subplot(232)
```

```
plt.scatter(x.loc[:, 'Avg. Area House Age'], y)
```

```
plt.title('Price VS House Age')
```

```
.....
```

```
plt.show()
```



实战2-2：多因子房价预测

```
#检查数据格式
```

```
print(x.shape)
```

```
(5000, 5)
```

```
# 3. 建构并训练模型
```

```
lr = LinearRegression()#使用LinearRegression构建线性回归模型
```

```
lr.fit(x_train,y_train)#训练模型
```

```
# 4. 查看模型参数
```

```
print('特征系数a为:',lr.coef_)
```

```
print('截距b为:',lr.intercept_)
```

```
特征系数a为: [ 2.16311170e+01 -1.66262626e+05  1.23736170e+05  1.51327027e+01  
-8.62595795e+01]
```

```
截距b为: -813067.2915261902
```

实现基本步骤

- 1、导入所需的库
- 2、准备数据集
- 3、建构并训练模型
- 4、查看模型参数
- 5、评估模型性能
- 6、可视化模型与数据点
- 7、预测新数据

实战2-2：多因子房价预测

5. 评估模型性能

```
y_predict=lr.predict(x)
```

```
print(y_predict)
```

```
MSE = mean_squared_error(y,y_predict)
```

```
R2 = r2_score(y,y_predict)
```

```
print(MSE, R2)
```

```
[1223968.89166086 1497306.3318863 1250884.31019437 ... 1020693.92390376
 1260503.36914585 1302737.7915763 ]
10219846512.17786 0.9180229195220739
```

6. 可视化模型与数据点

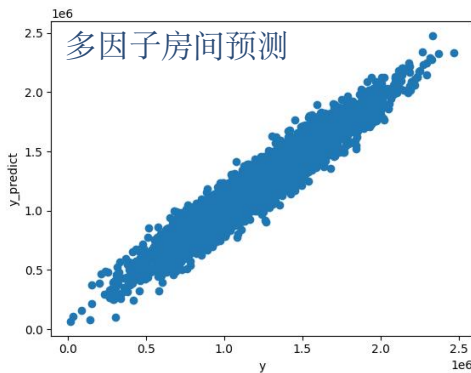
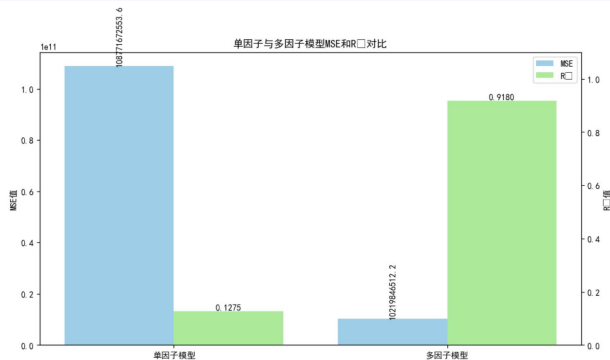
```
fig2 = plt.figure()
```

```
plt.scatter(y,y_predict)
```

```
plt.xlabel('y')
```

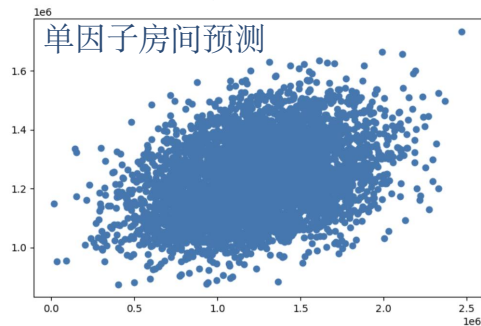
```
plt.ylabel('y_predict')
```

```
plt.show()
```



实现基本步骤

- 1、导入所需的库
- 2、准备数据集
- 3、建构并训练模型
- 4、查看模型参数
- 5、评估模型性能
- 6、可视化模型与数据点
- 7、预测新数据



1. 和一般的线性回归、多项式回归相比，岭回归（Ridge回归）和Lasso回归的优势是什么？
2. 如果用机器学习方法解决下图所示的问题，应该选择（ ）模型进行训练。

A、线性回归

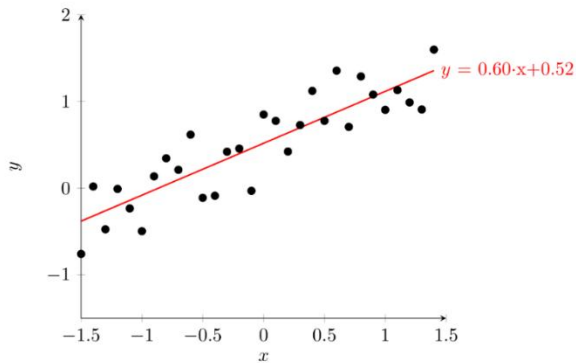
B、逻辑回归

下表给出了莫纳罗亚山（夏威夷岛的活火山）从1970年到2005年每5年的二氧化碳浓度，单位是百万分比浓度（Parts Per Million, ppm）。

年份(x)	1970	1975	1980	1985	1990	1995	2000	2005
CO ₂ (y)	325.68	331.15	338.69	345.90	354.19	360.88	369.48	379.67

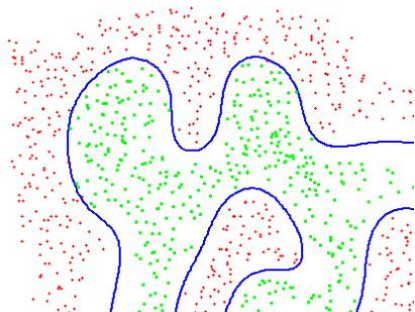
问题：1) 给出1984年二氧化碳浓度值；2) 预测2010年二氧化碳浓度值

- 线性回归处理的是连续的数值问题，预测出的结果是数值，例如房价和温度等。
- 逻辑回归（线性回归的扩展）属于分类算法，可以处理二元分类以及多元分类，预测结果是离散的分类，例如判断这封邮件是否是垃圾邮件，以及用户是否会点击此广告等。



线性回归

知道了价格后，能否预测客户是否购买（是/否）？

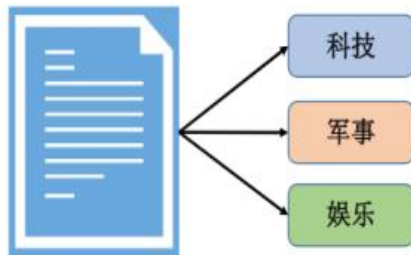


二分类问题

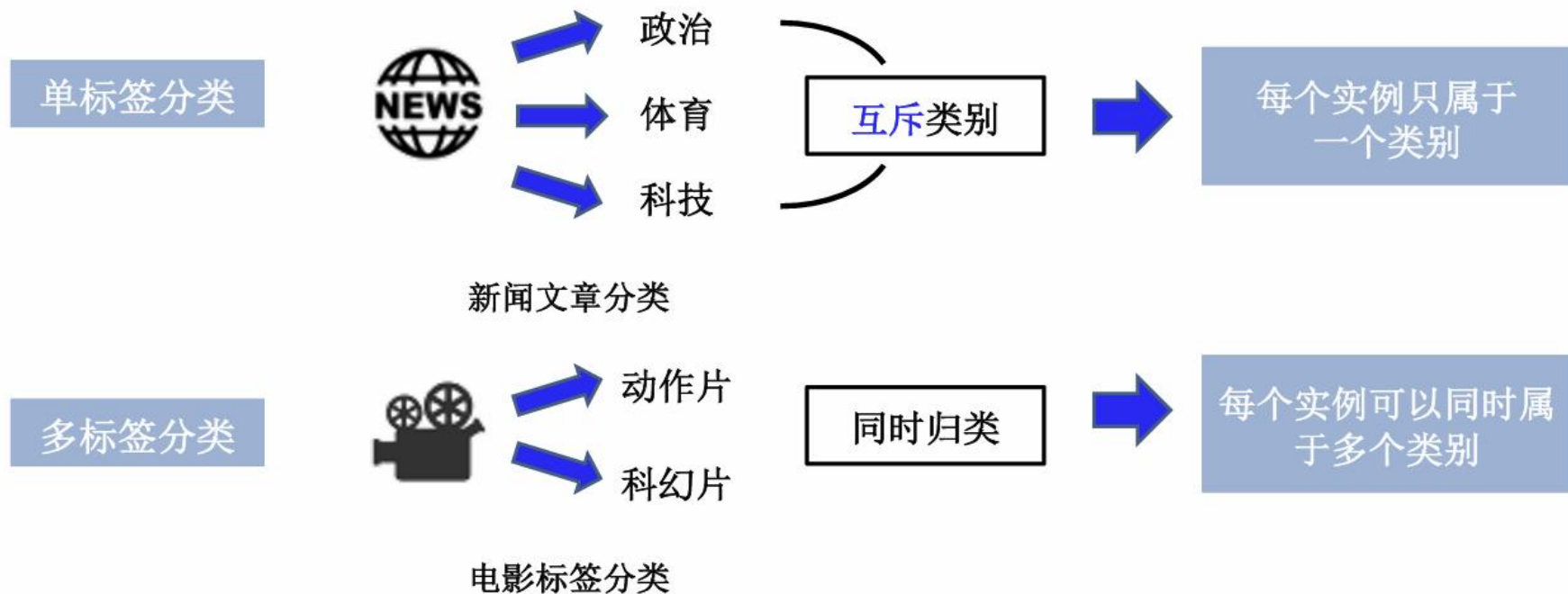
✓ 二分类问题



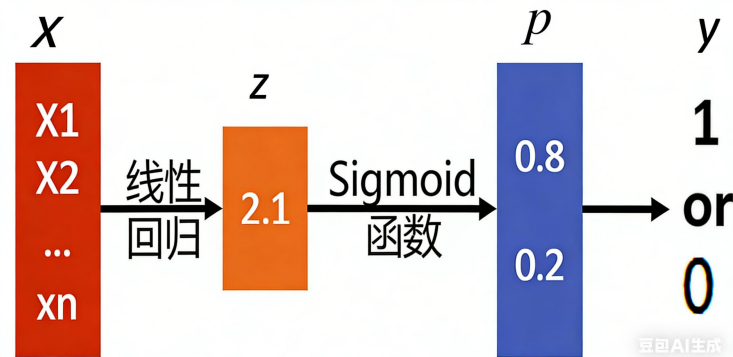
✓ 多分类问题



根据数据实例可属于的类别标签数量:

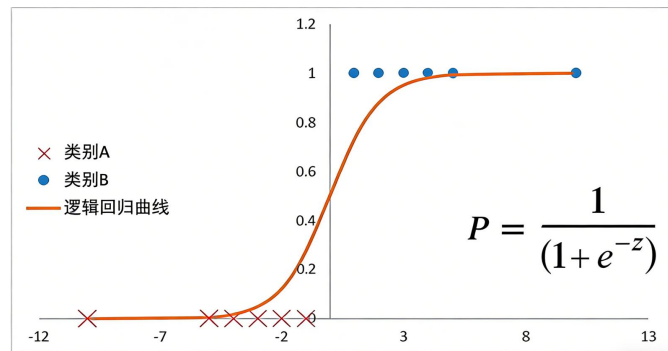


x1 年龄	x2 收入	是否 购买
28	1.2	1
24	0.5	0
...	...	...



- 1. 数据集 $D = \{(x_1, x_2, y)^1, (x_1, x_2, y)^2, \dots\}$ ，划分为训练集、验证集和测试集，
- 2. 模型 $f(x) \approx y$ ：逻辑回归 = 线性回归 + Sigmoid 函数 (Logistic 函数)，Sigmoid 函数将数值结果 $(-\infty, \infty)$ 转化为 0 到 1 之间的概率 p ；
- 3. 分类规则：根据概率 p 做预测，例如概率 p 大于阈值 0.5，则购买等等。

$$\text{Sigmoid}(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}$$



2

逻辑回归：二分类

• 4. 模型训练

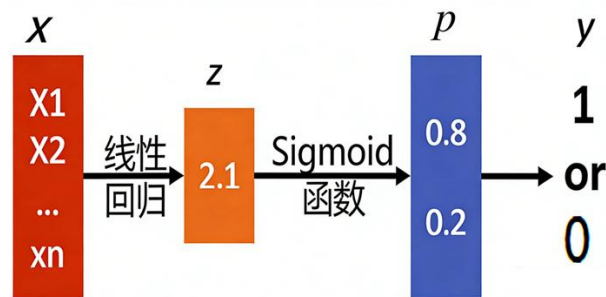
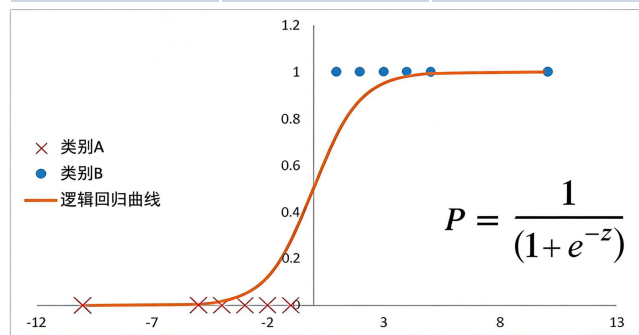
采用交叉熵损失函数衡量模型预测概率分布与真实标签分布之间的差异

$$L(w) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

$$= -\frac{1}{N} \sum_{i=1}^N [y_i \log\left(\frac{1}{1 + e^{-z}}\right) + (1 - y_i) \log\left(\frac{1}{1 + e^z}\right)]$$

使用梯度下降法不断调整参数 w_0, w_1, w_2 ，以最小化损失函数 $L(w)$ 。

x1 年龄	x2 收入	是否 购买
28	1.2	1
24	0.5	0
...	...	...



$$z = w_0 + w_1 x_1 + w_2 x_2$$

• 5. 模型测试

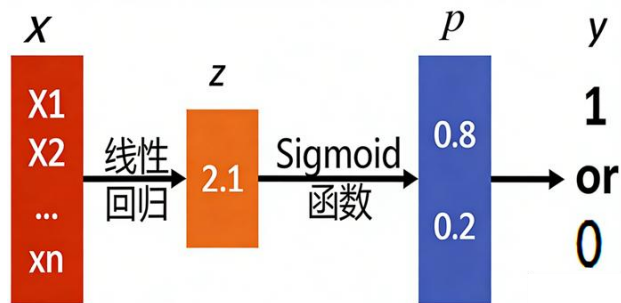
例如，对于客户年龄为35，收入为1.8万元，计算预测概率：

$$z = w_0 + w_1 \times 35 + w_2 \times 1.8$$

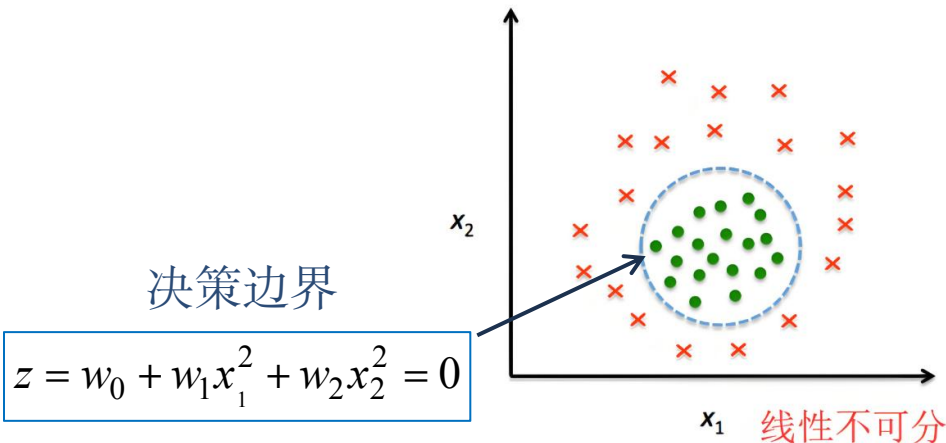
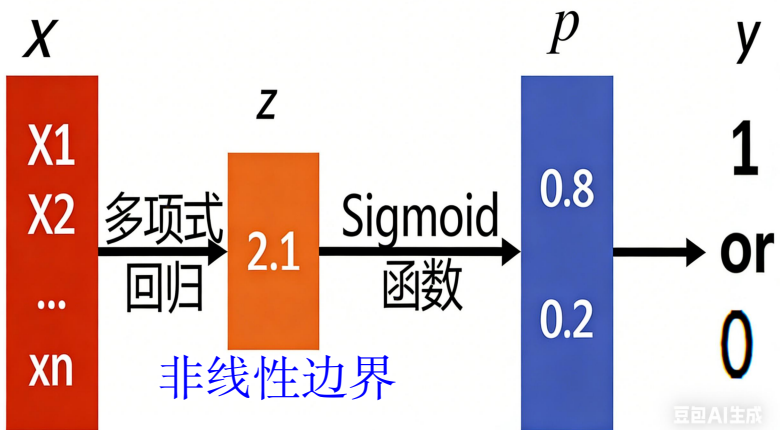
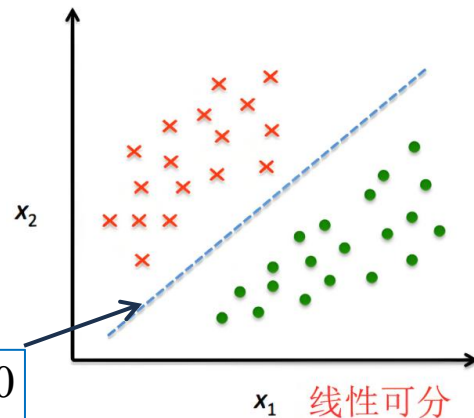
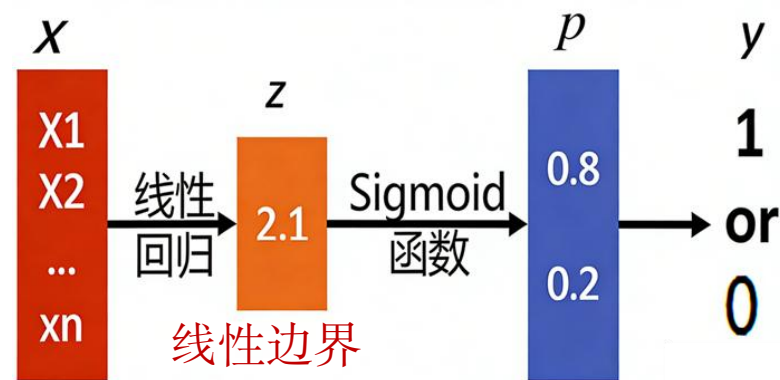
$$p = \text{sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

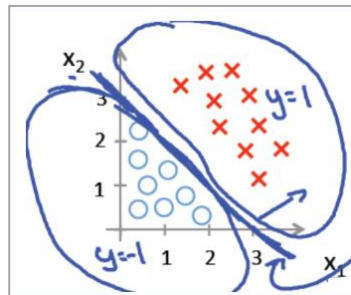
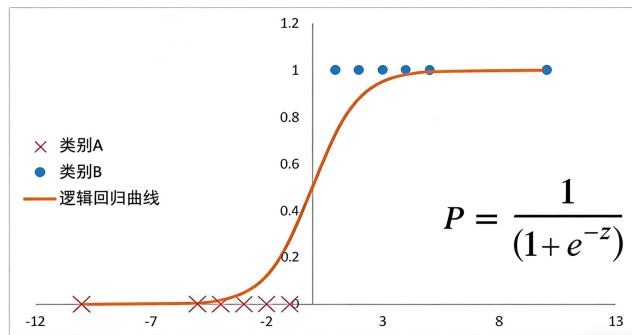
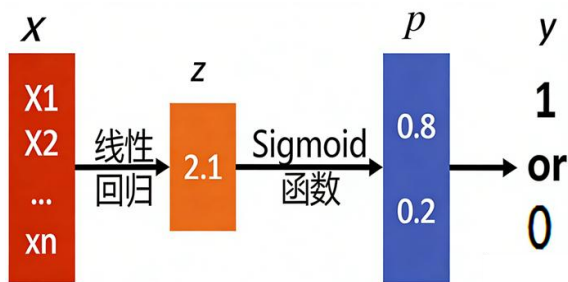
如果预测概率 p 大于阈值，则购买；
若否，则不购买。

x1 年龄	x2 收入	是否 购买
28	1.2	1
24	0.5	0
...	...	...



$$z = w_0 + w_1 x_1 + w_2 x_2$$





- 逻辑(Logistic)值 $z = w_0 + w_1x_1 + \dots + w_nx_n = \sum_{i=1}^n w_ix_i = \mathbf{w}^T \mathbf{x}$

- 分类函数/逻辑函数: $\text{sigmoid}(\mathbf{w}^T \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$

- 正样本的概率 $P(y = 1 | x) = \text{sigmoid}(\mathbf{w}^T \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$

- 负样本的概率 $P(y = 0 | x) = 1 - P(y = 1 | x) = \frac{1}{1 + e^{\mathbf{w}^T \mathbf{x}}}$

所有样本的概率越大越好!

w_0 : 截距/偏置

$w_1, \dots, w_n$: 特征系数/权重

$w_0, w_1, \dots, w_n$: 模型参数

$\mathbf{w}^T \mathbf{x}$ 越大越好

$\mathbf{w}^T \mathbf{x}$ 越小越好

对于正样本($y=1$), $\mathbf{w}^\top \mathbf{x}$ 大越好

对于负样本($y=0$), $\mathbf{w}^\top \mathbf{x}$ 小越好

所有样本 $y\mathbf{w}^\top \mathbf{x}$ 越大越好, 即所有样本概率越大越好!

$$P(y=1|x) = \text{sigmoid}(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

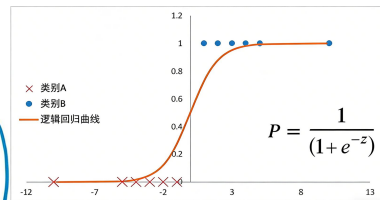
$$P(y=0|x) = 1 - P(y=1|x) = \frac{1}{1 + e^{\mathbf{w}^\top \mathbf{x}}}$$

◆ 最大似然函数作为逻辑回归模型的损失函数 (凸函数), 易得到参数的最优解, 其中似然函数就是所有样本的概率乘积

◆ 最大似然估计: 利用已知的样本结果信息, 反推最具有可能 (最大概率) 导致这些样本结果出现的模型参数值

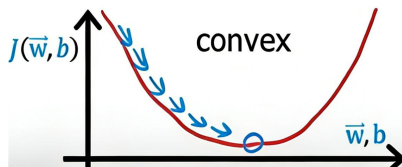
Squared error cost

$$J(\bar{\mathbf{w}}, b) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (f_{\bar{\mathbf{w}}, b}(\bar{\mathbf{x}}^{(i)}) - y^{(i)})^2$$



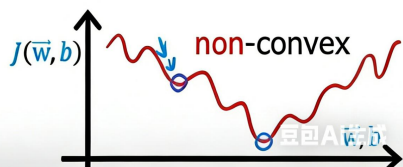
linear regression

$$f_{\bar{\mathbf{w}}, b}(\bar{\mathbf{x}}) = \bar{\mathbf{w}} \cdot \bar{\mathbf{x}} + b$$



logistic regression

$$f_{\bar{\mathbf{w}}, b}(\bar{\mathbf{x}}) = \frac{1}{1 + e^{-(\bar{\mathbf{w}} \cdot \bar{\mathbf{x}} + b)}}$$



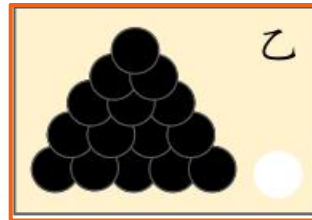
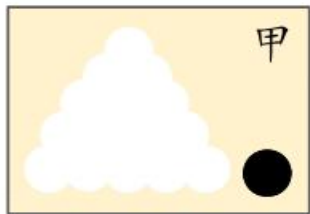
猜猜：谁最有可能击中这只兔子？

- ☐ A 孩子
- ☐ B 猎人
- ☐ C 不知道



提交

- 例：有两个外形完全相同的箱子，甲箱中有99只白球，1只黑球；乙箱中有99只黑球，1只白球。一次试验取出一球，结果取出的是黑球。



- 问：黑球从哪个箱子中取出？
- 第一印象就是：“此黑球最像是从乙箱中取出的”，这个推断符合人们的经验事实。“最像”就是“最大似然”之意，这种想法常称为“最大似然原理”(maximum-likelihood)。

□ 似然函数 (likelihood function) :
即所有样本同时出现的联合概率
= 每个样本概率的乘积

$$P(x_1, x_2, \dots, x_N | w) = \prod_{i=1}^N P(x_i | w)$$

N个独立的
训练样本

模型
参数

【例2.2】假设一个不透明盒里装有3颗围棋子，现用有放回抽样法随机抽取三次，每次拿一颗，得到白子2次黑子1次。试用最大似然估计法估计盒中白子个数。

【解】设盒中有 θ ($\theta = 0, 1, 2, 3$)枚白子，则有：

当 $\theta = 0$ 时， $p(\text{白}|\theta) = 0$ ； 当 $\theta = 1$ 时， $p(\text{白}|\theta) = 1/3$ ；

当 $\theta = 2$ 时， $p(\text{白}|\theta) = 2/3$ ； 当 $\theta = 3$ 时， $p(\text{白}|\theta) = 1$ 。

三次采样中抽到了2次白子1次黑子，故似然函数为：

$$L(\theta) = [p(\text{白}|\theta)]^2[1 - p(\text{白}|\theta)]。$$

分别取 $\theta = 0, 1, 2, 3$ ，可得：

$$L(0) = 0, \quad L(1) = 2/27, \quad L(2) = 4/27, \quad L(3) = 0。$$

为使得事件“三次采样抽中2次白子1次黑子”发生概率最大，应取 $\hat{\theta} = 2$ 作为参数 θ 的最大似然估计。

2

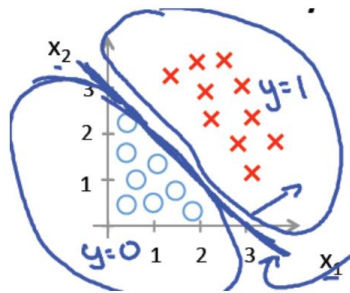
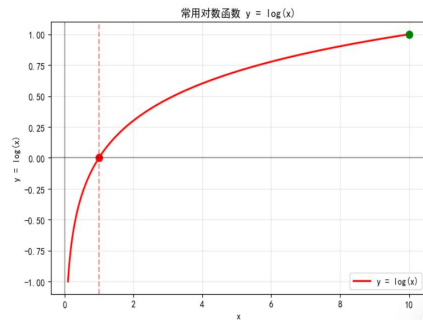
逻辑回归——二分类 (0/1分类)

$$\frac{1}{N} \sum_i -[y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i)]$$

最大化似然函数: $\max \prod_{i=1}^N P(y_i | x_i, w)$

二分类问题: $P(y=1 | x) = \text{Sigmoid}(w^T x) = \frac{1}{1 + e^{-w^T x}}, y=1$

$$P(y=0 | x) = 1 - P(y=1 | x) = \frac{1}{1 + e^{w^T x}}, y=0$$



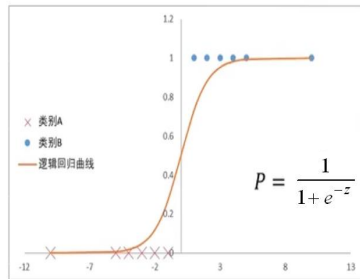
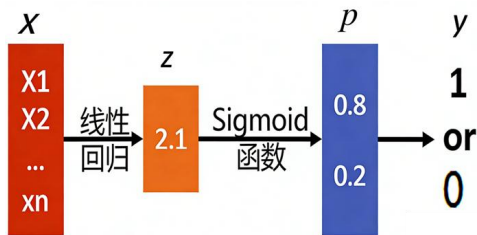
所有样本的最大化似然函数: $\max \prod_{i=1}^N \left(\frac{1}{1 + e^{-w^T x_i}} \right)^{y_i} \left(\frac{1}{1 + e^{w^T x_i}} \right)^{(1-y_i)}$

对上述函数取对数: $\max \sum_{i=1}^N [y_i \log\left(\frac{1}{1 + e^{-w^T x_i}}\right) + (1 - y_i) \log\left(\frac{1}{1 + e^{w^T x_i}}\right)]$

最小化代价函数: $\min -\frac{1}{N} \sum_{i=1}^N [y_i \log\left(\frac{1}{1 + e^{-w^T x_i}}\right) + (1 - y_i) \log\left(\frac{1}{1 + e^{w^T x_i}}\right)]$

经验风险: 平均代价/
二元交叉熵损失函数

$$\min \frac{1}{N} \sum_{i=1}^N -[y_i \log\left(\frac{1}{1 + e^{-w^T x_i}}\right) + (1 - y_i) \log\left(\frac{1}{1 + e^{w^T x_i}}\right)]$$



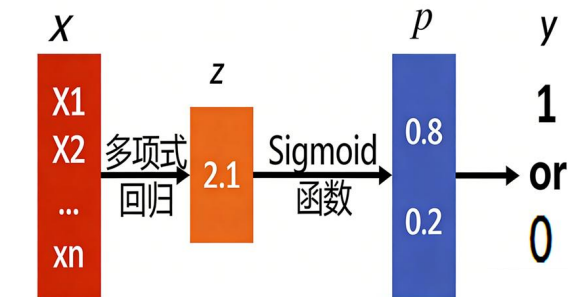
$$J(w) = \underbrace{-\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]}_{\text{经验风险: 二元交叉熵损失函数}} + \underbrace{\frac{\lambda}{2} \|w\|^2}_{\text{L2正则}}$$

期望
风险

经验风险: 二元交叉熵损失函数

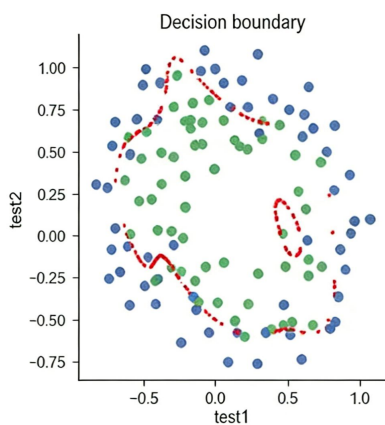
L2正则

结构
风险

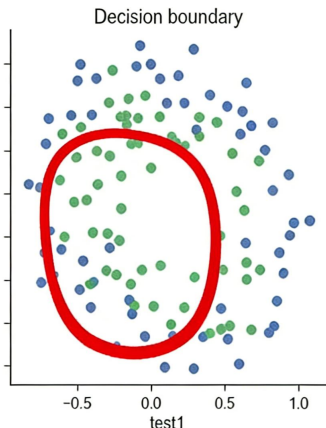


$$P(y = 1 | x) = \frac{1}{1 + e^{-w^T x}}$$

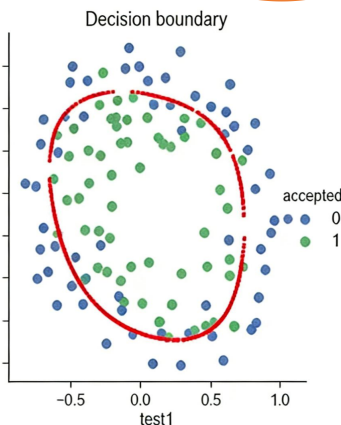
$$P(y = 0 | x) = 1 - P(y = 1 | x)$$



没有正则化, 过拟合



正则化过度, 欠拟合

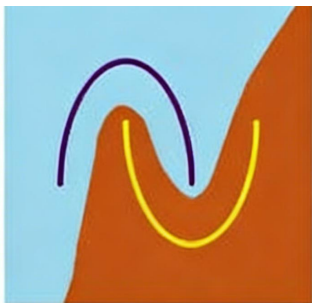
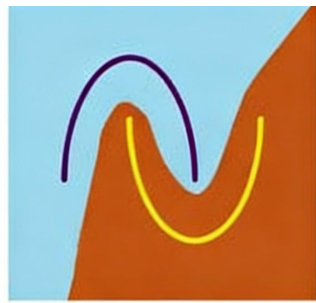
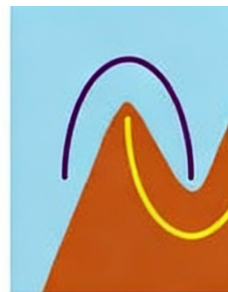


适当的正则化

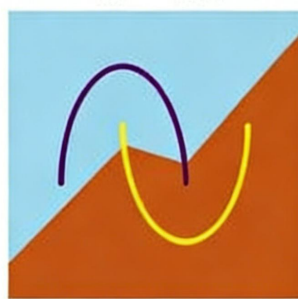
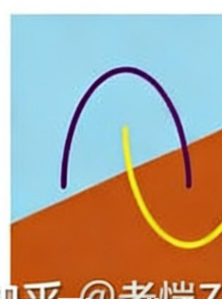
accepted
0
1

$$J(w) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] + \frac{\lambda}{2} \|w\|^2$$

期望
风险


 $\lambda = 0$

 $\lambda = 0.1$

 $\lambda = 1$

 $\lambda = 2.5$

 $\lambda = 5$

 $\lambda = 7.5$

 $\lambda = 10$

λ 值变大，权重的值趋近于0，决策边界逐渐变成折线，最后成为一条直线

训练问题:

$$\operatorname{argmin}_{\mathbf{w}} J(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

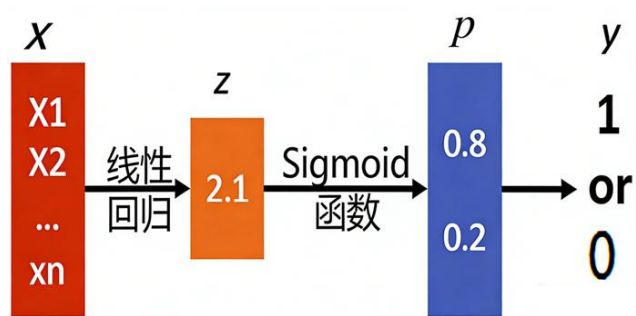
梯度(向量化表示):

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{N} (\mathbf{p} - \mathbf{y})^T \mathbf{x} + \lambda \mathbf{w}$$

参数更新:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}=\mathbf{w}_t} J(\mathbf{w})$$

学习率



$$P(y = 1|x) = \text{Sigmoid}(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$$

- ✓ 对于k类问题, 第一种方案:
 - 训练k个one vs. the rest ('ovr' 一对其余) 分类器 (二分类问题)
 - 山鸢尾-非山鸢尾分类器
 - 变色鸢尾-非变色鸢尾分类器
 - 维吉尼亚鸢尾-非维吉尼亚鸢尾分类器
 - 综合k个分类器的结果判别, 未知样本的预测结果取概率最大的类别
- ✓ 第二种方案: 直接训练多分类的分类器 ('multinomial' “多项式逻辑回归”或 Softmax回归)



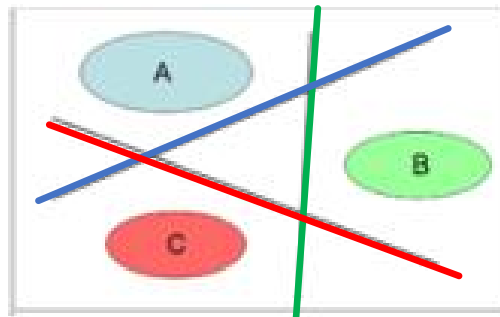
山鸢尾



变色鸢尾



维吉尼亚鸢尾

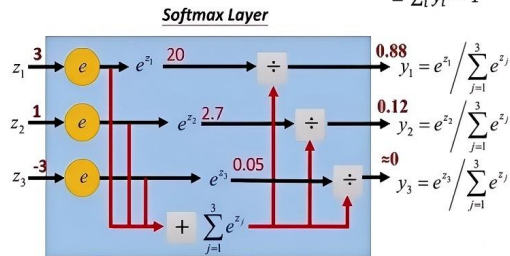


• Softmax layer as the output layer

Probability:

■ $1 > y_i > 0$

■ $\sum_i y_i = 1$



逻辑回归模型中，Sigmoid函数的作用是什么？

- ☐ A 将数值结果转化为0到1之间的概率
- ☐ B 将数值结果转化为-1到1之间的概率
- ☐ C 将数值结果转化为0到100之间的概率
- ☐ D 将数值结果转化为 $-\infty$ 到 ∞ 之间的概率

提交

逻辑回归模型中，Sigmoid函数的作用是什么？

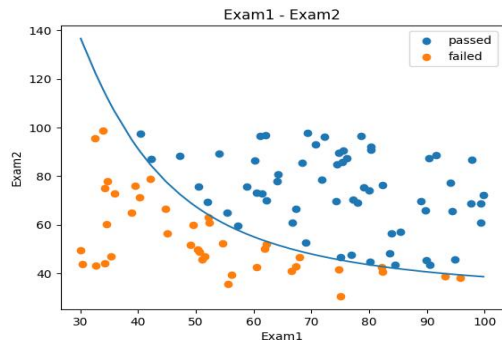
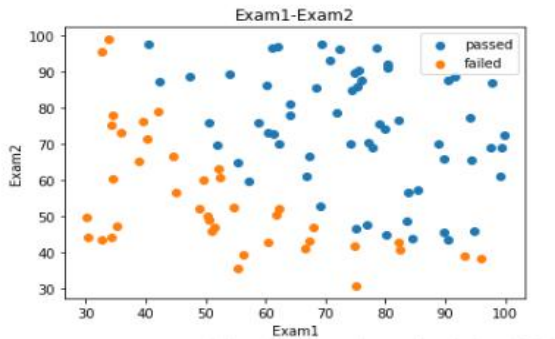
- ☒ A 将数值结果转化为0到1之间的概率
- ☐ B 将数值结果转化为-1到1之间的概率
- ☐ C 将数值结果转化为0到100之间的概率
- ☐ D 将数值结果转化为 $-\infty$ 到 ∞ 之间的概率

提交

逻辑回归:考试通过预测

- 基于examdata.csv数据, 建立逻辑回归模型 (步骤和线性回归基本相同);
预测Exam1 = 70, Exam2 = 60时, 该同学是通过还是不通过 passed or failed;
建立二阶边界, 提高模型准确度。

	Exam1	Exam2	Pass
0	34.623660	78.024693	0
1	30.286711	43.894998	0
2	35.847409	72.902198	0
3	60.182599	86.308552	1
4	79.032736	75.344376	1



实战1-1:考试通过预测

#1、导入所需的库

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from matplotlib import pyplot as plt
import numpy as np
import pandas as pd
```

#2、准备数据集

```
data = pd.read_csv('examdata.csv')
data.head()
```

	Exam1	Exam2	Pass
0	34.623660	78.024693	0
1	30.286711	43.894998	0
2	35.847409	72.902198	0
3	60.182599	86.308552	1
4	79.032736	75.344376	1

实战1-1:考试通过预测

#可视化

fig1 = plt.figure()

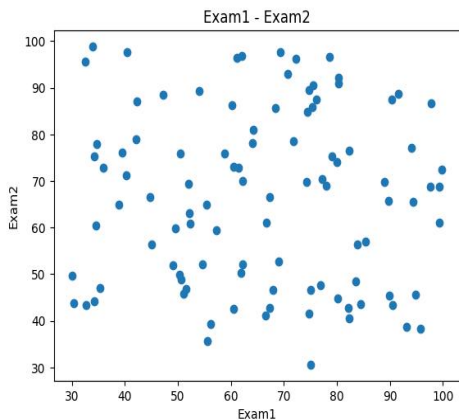
plt.scatter(data.loc[:, 'Exam1'], data.loc[:, 'Exam2'])

plt.title('Exam1 - Exam2')

plt.xlabel('Exam1')

plt.ylabel('Exam2')

plt.show()



实战1-1:考试通过预测

#可视化

fig2 = plt.figure()

mask = data.loc[:, 'Pass'] == 1

passed = plt.scatter(data.loc[:, 'Exam1'][mask], data.loc[:, 'Exam2'][mask], label='passed')

failed = plt.scatter(data.loc[:, 'Exam1'][~mask], data.loc[:, 'Exam2'][~mask], label='failed')

plt.legend()

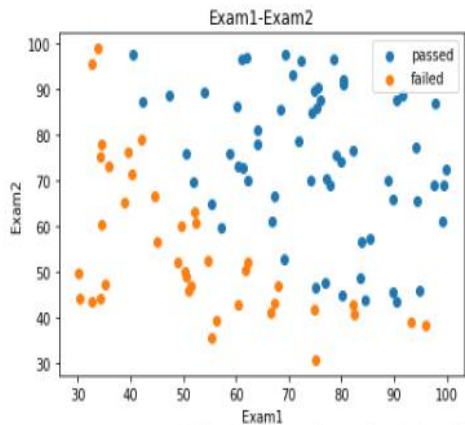
plt.title('Exam1 - Exam2')

plt.xlabel('Exam1')

plt.ylabel('Exam2')

plt.show()

	Exam1	Exam2	Pass
0	34.623660	78.024693	0
1	30.286711	43.894998	0
2	35.847409	72.902198	0
3	60.182599	86.308552	1
4	79.032736	75.344376	1



实战1-1:考试通过预测

#3、构建并训练模型

```
x = data.drop(['Pass'],axis=1)
```

```
y = data.loc[:, 'Pass']
```

```
LR = LogisticRegression()
```

```
LR.fit(x,y) #无返回值, 但会修改模型本身的  
参数 (model.coef_ 权重, model.intercept_ 偏置)
```

	Exam1	Exam2	Pass
0	34.623660	78.024693	0
1	30.286711	43.894998	0
2	35.847409	72.902198	0
3	60.182599	86.308552	1
4	79.032736	75.344376	1

LogisticRegression(

`penalty='l2'`, # 正则化方式

`dual=False`, # 是否使用对偶形式 (仅适用于l2正则化和liblinear solver)

`tol=1e-4`, # 收敛阈值 (迭代停止条件)

`C=1.0`, # 正则化强度的倒数 (值越小, 正则化越强)

`fit_intercept=True`, # 是否拟合截距项 (偏置)

`intercept_scaling=1`, # 截距项的缩放因子 (仅对liblinear solver有效)

`class_weight=None`, # 类别权重 (处理不平衡数据)

`random_state=None`, # 随机数种子 (用于随机初始化或打乱数据)

`solver='lbfgs'`, # 优化求解器

`max_iter=100`, # 最大迭代次数

`multi_class='auto'`, # 多分类策略

`verbose=0`, # 日志输出详细程度

`warm_start=False`, # 是否使用上一次训练的结果作为初始值

`n_jobs=None`, # 并行计算的线程数

`l1_ratio=None` # L1正则化的比例 (仅用于elasticnet penalty)

实战1-1:考试通过预测

4. 预测并评估模型性能

```
y_predict = LR.predict(x) #输出每个样本的预测类别, 阈值默认为0.5
```

```
accuracy = accuracy_score(y, y_predict)
```

```
print(accuracy) #0.89
```

```
y_test = LR.predict([[70,65]])
```

```
print(y_test) #[1]
```

	Exam1	Exam2	Pass
0	34.623660	78.024693	0
1	30.286711	43.894998	0
2	35.847409	72.902198	0
3	60.182599	86.308552	1
4	79.032736	75.344376	1

准确率accuracy:

- 正确预测的样本数占总样本数的比例, 是最常用的评估指标之一
- 准确率适用于正负样本分布均衡的场景, 若数据不平衡 (如 99% 是负类), 则需结合精确率 (Precision)、召回率 (Recall) 等指标综合评估

实战1-1: 考试通过预测

#5、绘制决策边界

#决策边界参数, $z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$

`theta0 = lr.intercept_`

`theta1, theta2 = lr.coef_[0][0], lr.coef_[0][1]`

`print(theta0, theta1, theta2)` `[-25.05219314] 0.20535491217790397 0.2005838039546906`

#根据x1生成x2_new

`x1 = x.loc[:, 'Exam1']`

`x2_new = -(theta0+theta1*x1)/theta2`

```
[34.62365962 30.28671077 35.84740877 60.18259939 79.03273605 45.08327748
61.10666454 75.02474557 76.0987867 84.43281996 95.86155507 75.01365839
82.30705337 69.36458876 39.53833914 53.97105215 69.07014406 67.94685548
70.66150955 76.97878373 67.37202755 89.67677575 50.53478829 34.21206098
77.92409145 62.27101367 80.19018075 93.1143888 61.83020602 38.7858038
61.37928945 85.40451939 52.10797973 52.04540477 40.23689374 54.63510555
33.91550011 64.17698887 74.78925296 34.18364003 83.90239366 51.54772027
94.44336777 82.36875376 51.04775177 62.22267576 77.19303493 97.77159928
62.0730638 91.5649745 79.94481794 99.27252693 90.54671411 34.52451385
50.28649612 49.58667722 97.64563396 32.57720017 74.24869137 71.79646206
75.39561147 35.28611282 56.2538175 30.05882245 44.66826172 66.56089447
40.45755098 49.07256322 80.27957401 66.74671857 32.72283304 64.03932042
72.34649423 60.45788574 58.84095622 99.8278578 47.26426911 50.4581598
60.45555629 82.22666158 88.91389642 94.83450672 67.31925747 57.23870632
80.366756 68.46852179 42.07545454 75.47770201 78.63542435 52.34800399
94.09433113 90.44855097 55.48216114 74.49269242 89.84580671 83.48916274
42.26170081 99.31500881 55.34001756 74.775893 ]
```

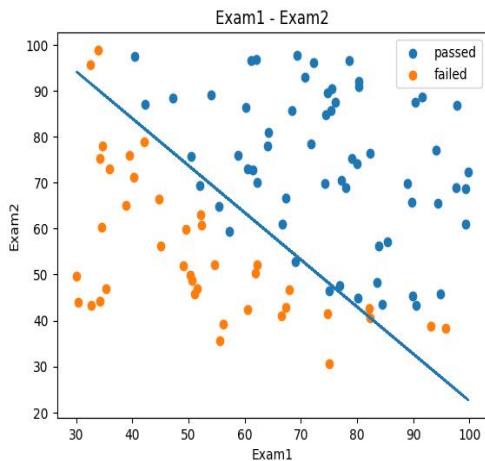
0	34.623660
1	30.286711
2	35.847409
3	60.182599
4	79.032736
	...
95	83.489163
96	42.261701
97	99.315009
98	55.340018
99	74.775893

实战1-1:考试通过预测

#5、绘制决策边界（线性边界）

```
fig3 = plt.figure()
plt.plot(x1, x2_new)
passed = plt.scatter(data.loc[:, 'Exam1'][mask], data.loc[:, 'Exam2'][mask])
failed = plt.scatter(data.loc[:, 'Exam1'][~mask], data.loc[:, 'Exam2'][~mask])

plt.title('Exam1 - Exam2')
plt.xlabel('Exam1')
plt.ylabel('Exam2')
plt.legend((passed, failed), ('passed', 'failed'))
plt.show()
```



在逻辑回归模型中，准确率（accuracy）是指什么？

- ☐ A 模型预测为正类的样本中，实际为正类的比例
- ☐ B 所有实际为正类的样本中，被正确预测的比例
- ☐ C 正确预测的样本数占总样本数的比例
- ☐ D 模型预测结果的方差大小

提交

在逻辑回归模型中，准确率（accuracy）是指什么？

- ☐ A 模型预测为正类的样本中，实际为正类的比例
- ☐ B 所有实际为正类的样本中，被正确预测的比例
- ☒ C 正确预测的样本数占总样本数的比例
- ☐ D 模型预测结果的方差大小

提交

实战1-2:考试通过预测 (多项式边界) $z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2 = 0$

#生成新多项式 (二次项) 特征数据

$x1_2 = x1 * x1$; $x2_2 = x2 * x2$; $x1_x2 = x1 * x2$

$x_new = \{ 'x1':x1, 'x2':x2, 'x1_2':x1_2, 'x2_2':x2_2, 'x1_x2':x1_x2 \}$

$x_new = \text{pd.DataFrame}(x_new)$

$\text{print}(x_new)$

#训练新模型, 评估新模型性能

$\text{LR2} = \text{LogisticRegression}()$

$\text{LR2.fit}(x_new, y)$

$y2_predict = \text{LR2.predict}(x_new)$

$\text{accuracy2} = \text{accuracy_score}(y, y2_predict)$

$\text{print}(\text{accuracy2}) \quad \#1.0$

	X1	X2	X1_2	X2_2	X1_X2
0	34.623660	78.024693	1198.797805	6087.852690	2701.500406
1	30.286711	43.894998	917.284849	1926.770807	1329.435094
2	35.847409	72.902198	1285.036716	5314.730478	2613.354893
3	60.182599	86.308552	3621.945269	7449.166166	5194.273015
4	79.032736	75.344376	6246.173368	5676.775061	5954.672216
..	...	...	...	...	...
95	83.489163	48.380286	6970.440295	2340.652054	4039.229555
96	42.261701	87.103851	1786.051355	7587.080849	3681.156888
97	99.315009	68.775409	9863.470975	4730.056948	6830.430397
98	55.340018	64.931938	3062.517544	4216.156574	3593.334590
99	74.775893	89.529813	5591.434174	8015.587398	6694.671710

[100 rows x 5 columns]

实战1-2: 考试通过预测 (多项式边界) $z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2 = 0$

#换一个方法, 产生多项式特征数据

from `sklearn.preprocessing` import `PolynomialFeatures`

degree=2表示创建最高2次幂的多项式特征

poly = `PolynomialFeatures`(degree=2, include_bias= True)

X_train_poly = `poly.fit_transform`(X)

#establish new model and train

LR2 = `LogisticRegression`(max_iter=500)

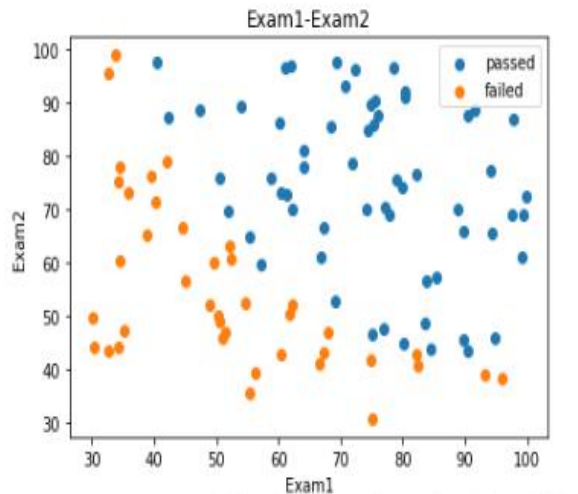
LR2.`fit`(X_train_poly, y)

预测并计算准确确定

y2_predict = LR2.`predict`(X_train_poly)

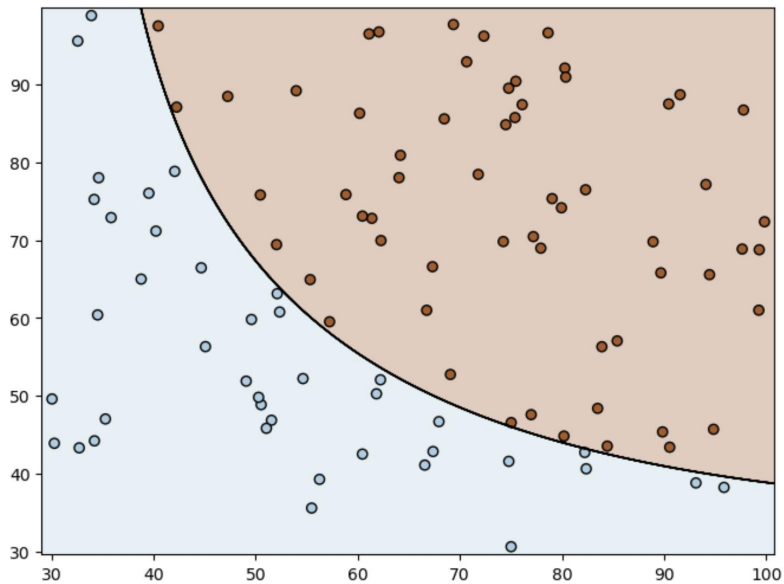
accuracy2 = `accuracy_score`(y, y2_predict)

print(accuracy2) #1.0

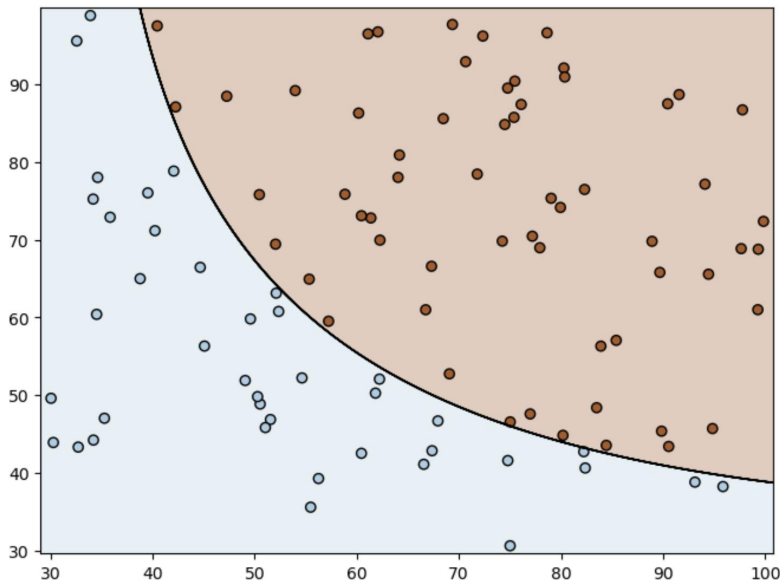


实战1-2:考试通过预测（多项式边界） $z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2 = 0$

```
# 函数定义，绘制决策边界
def plot_decision_boundary(X, y, model, poly, ax=None):
    if ax is None: ## 如果未指定绘图轴，则用当前默认的轴
        ax = plt.gca()
    # 确定绘图范围
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    # 生成网格点
    h = 0.02 # 网格步长
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                          np.arange(y_min, y_max, h))
```



实战1-2:考试通过预测（多项式边界） $z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2 = 0$



将网格点转换为二次项特征并预测

```
grid_points = np.c_[xx.ravel(), yy.ravel()] #按列拼接成特征矩阵
```

```
grid_poly = poly.transform(grid_points) #转换为高次项特征
```

```
Z = model.predict(grid_poly)
```

```
Z = Z.reshape(xx.shape) #重塑为二维形状
```

绘制不同类别区域和决策边界

```
ax.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.Paired)
```

```
ax.contour(xx, yy, Z, linewidths=1, colors='black')
```

绘制原始数据点

```
ax.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired,  
           edgecolors='k')
```

实战1-2:考试通过预测（多项式边界） $z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2 = 0$

#数据格式转换

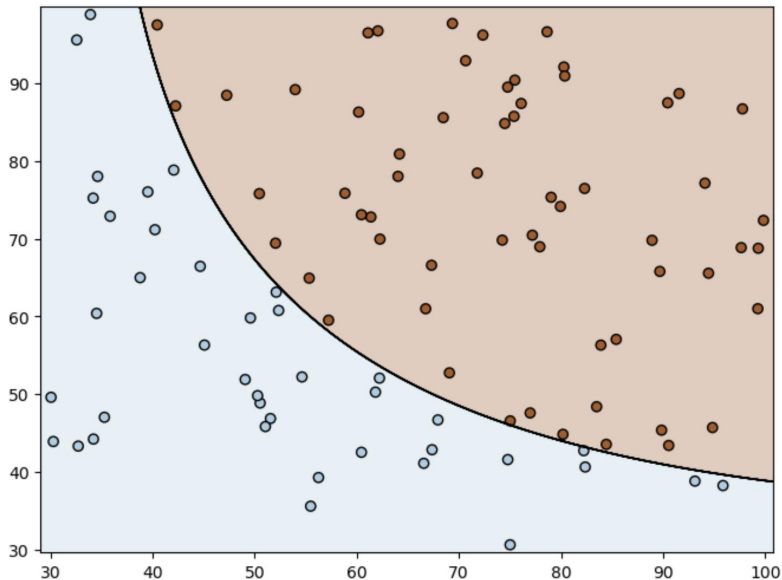
```
x_array = x.to_numpy()
```

#可视化，函数调用

```
fig3=plt.figure(figsize=(8, 6))
```

```
plot_decision_boundary(x_array , y, LR2, poly)
```

```
plt.show()
```



实战1-3:考试通过预测（过拟合）

#划分训练集和测试集

```
from sklearn.model_selection import train_test_split
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=42)
```

#生成多项式（6次项）特征数据：

```
from sklearn.preprocessing import PolynomialFeatures
```

```
poly6=PolynomialFeatures(degree=6) #生成的最高次
```

```
x_train_6=poly6.fit_transform(x_train) #生成新的数据集，第一次使用fit_transform方法
```

```
x_test_6=poly6.transform(x_test)#以后就用transform方法
```

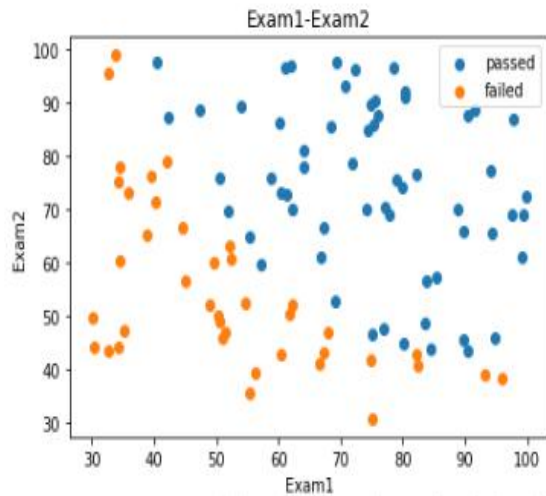
#训练新模型

```
LR6 = LogisticRegression()
```

```
LR6.fit(x_train_6,y_train)
```

实战1-3:考试通过预测 (过拟合)

#评估新模型性能

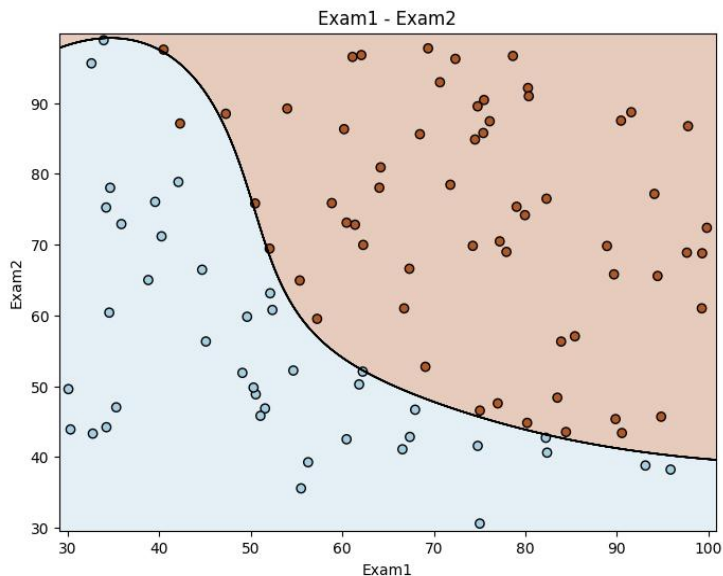
`y_train_6_predict = LR6.predict(x_train_6)``accuracy_train_6 = accuracy_score(y_train,y_train_6_predict)``y_test_6_predict = LR6.predict(x_test_6)``accuracy_test_6 = accuracy_score(y_test,y_test_6_predict)``print(accuracy_train_6, accuracy_test_6)``1.0 0.9666666666666667`

实战1-3:考试通过预测 (过拟合)

#数据格式转换

`x_array = x.to_numpy()`

#可视化, 函数调用

`fig3=plt.figure(figsize=(8, 6))``plot_decision_boundary(x_array, y, LR6, poly6)``plt.show()`

步骤	处理过程
准备数据	收集数据，进行数据预处理及提取特征并划分训练、验证和测试数据
选择模型	确定是回归模型还是分类模型
选择损失函数	回归问题常用均方误差，分类问题常用交叉熵
模型训练、验证	通过梯度下降法优化参数，最小化损失函数
模型测试	使用训练好的模型进行测试
模型评估	回归问题常用MSE、R2等进行性能评估，分类问题常用正确率Acc、F1分数等进行性能评估



- 如果用逻辑回归解决如下图所示的鸢尾花分类问题，应该怎么设计分类器？
(要求给出至少两种方案，每种方案都要求给出分类器的数量、性质和分类函数)

问题：如何根据鸢尾花的花萼和花瓣大小将其分为三种不同的品种？



山鸢尾



变色鸢尾



维吉尼亚鸢尾

特征	花萼长度	花萼宽度	花瓣长度	花瓣宽度	model →	结果	品种 (标签)
	5.1	3.3	1.7	0.5			0 (山鸢尾)
	5.0	2.3	3.3	1.0			1 (变色鸢尾)
	6.4	2.8	5.6	2.2			2 (维吉尼亚鸢尾)

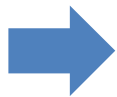
假如训练了一个分类的模型用来识别病人的癌症检测结果是否为阳性，现在用200张图片来验证模型的性能指标。这200张图片中，198张是阴性，2张是阳性。模型的识别结果的混淆矩阵如下表所示。

要求：计算识别结果的**正确率(Acc)**、**精度(P)**、**召回率(R)**、**F1分数(F1)**、**敏感性 (Sen)** 和**特异性 (Spe)** 。

预测值 \ 真实值	阳性	阴性
阳性	1	1
阴性	1	197

学习内容

CONTENTS



1

机器学习概述

2

线性回归与逻辑回归

3

支持向量机

4

聚类算法

5

决策树

6

强化学习